



暨南大學
JINAN UNIVERSITY

Undergraduate Lab Report

Course Title: Experiment of Computer Organization

Course No: 60080014

Student Name: 李榮耀

Student No: 2024103975

School: International School

Department: _____

Major: CST

Instructor: SUN Heng

Academic Year: 2025~2026, Semester : 1st [☒] 2nd [☐]

Academic Affairs Office of Jinan University

Date (dd/mm/yyyy) 2025/12/3

Computer Organization Lab List

Student Name: Li Rongyao Student No:

2024103975

ID	Lab Name	Type
1	Number Storage Lab	Individual
2	Manipulating Bits	Individual
3	A Simple Real-life Control System Lab	Team
4	System I/O	Individual
5	Simulating Y86-64 Program	Individual
6	Performance Lab	Team

Undergraduate Lab Report of Jinan University

Course Title Experiment of computer organization

Evaluation

Lab Name Simulating Y86-64 Program

Instructor Sun Heng

Lab Address Room 116

Student Name Li Rongyao

Student No 2024103975

College International School

Department _____ Major CST

Date 2025/12/3 Afternoon

1. Introduction

- To learn about the implementation of a Y86-64 processor.
- To have a keen appreciation for the interactions between instruction and hardware that affect the programs.
- To run two simple Y86-64 programs and become familiar with the Y86-64 tools.

2. Lab Instructions or Steps

- Find Sources List:

Make sure the link in **sources.list** match your ubuntu version:

For Ubuntu 16.04 **xenial**:

[模板:16.04source - Ubuntu 中文](#)

- Backup: sources.list.bak

sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak

- Modify: Rewrite sources.list using source from the website.

```
# sudo gedit /etc/apt/sources.list
```

- Update:

```
# sudo apt-get update (upgrade / -f install)
```

```
# sudo apt-get install flex bison
```

For Ubuntu16.04:

```
# sudo apt-get install tcl8.5 tcl8.5-dev tk8.5 tk8.5-dev
```

For Ubuntu16.04:

In **sim** and **seq** directory, find **Makefile**, and modify it:

```
TKLIBS=-L/usr/lib -ltk8.5 -ltcl8.5|
# Modify the following line so that gcc
# header files on your system. Comment
# Tcl/Tk.
TKINC=-isystem /usr/include/tcl8.5
```

(1) Start by copying this lab to a directory in which you plan to do your work.

(2) Unpacked the file using:

```
# tar xvf sim.tar (Or just double click and choose extract)
```

The directory **sim** contains the following subdirectories:

misc Source code files for utilities such as YAS (the Y86-64 assembler), and YIS (the Y86-64 instruction set simulator). It also contains the isa.c source file that is used by all of the processor simulators.

y86-code Y86-64 assembly code for many of the example programs. As a running example, we will use the program `asum.js` in this

subdirectory. The compiled version of the program is shown in Figure 1.

```

1      | # Execution begins at address 0
2 0x000: | .pos 0
3 0x000: 30f400020000000000000000 | irmovq stack, %rsp      # Set up stack pointer
4 0x00a: 803800000000000000000000 | call main              # Execute main program
5 0x013: 00 | halt                  # Terminate program
6      |
7      | # Array of 4 elements
8 0x018: | .align 8
9 0x018: 0d000d000d000000 | array: .quad 0x000d000d000d
10 0x020: c000c000c0000000 | .quad 0x00c000c000c0
11 0x028: 000b000b000b0000 | .quad 0x0b000b000b00
12 0x030: 00a000a000a00000 | .quad 0xa000a000a000
13      |
14 0x038: 30f718000000000000000000 | main:  irmovq array,%rdi
15 0x042: 30f604000000000000000000 |      irmovq $4,%rsi
16 0x04c: 805600000000000000000000 |      call sum          # sum(array, 4)
17 0x055: 90 |      ret
18      |
19      | # long sum(long *start, long count)
20      | # start in %rdi, count in %rsi
21 0x056: 30f808000000000000000000 | sum:    irmovq $8,%r8      # Constant 8
22 0x060: 30f901000000000000000000 |      irmovq $1,%r9      # Constant 1
23 0x06a: 6300 |      xorq %rax,%rax      # sum = 0
24 0x06c: 6266 |      andq %rsi,%rsi      # Set CC
25 0x06e: 708700000000000000000000 |      jmp test          # Goto test
26 0x077: 50a700000000000000000000 | loop:   mrmovq (%rdi),%r10 # Get *start
27 0x081: 60a0 |      addq %r10,%rax      # Add to sum
28 0x083: 6087 |      addq %r8,%rdi      # start++
29 0x085: 6196 |      subq %r9,%rsi      # count--. Set CC
30 0x087: 747700000000000000000000 | test:   jne loop        # Stop when 0 #
31 0x090: 90 |      ret                # Return
32      |
33      | # Stack starts here and grows to lower addresses
34 0x200: | .pos 0x200
35 0x200: | stack:

```

Figure 1 Sample object code file. This code is in the file `asum.yo` in the `y86-code` subdirectory.

(3) Change to the **sim** directory and build the Y86-64 tools:

make clean

make

(4) For the Y86-64processor, the simulator can be run in TTY or GUI mode:

TTY mode Uses a minimalist, terminal-oriented interface. Prints everything on the terminal output. Not very convenient for

debugging but can be installed on any system and can be used for automated testing. The default mode for all simulators.

GUI mode Has a graphic user interface, to be described shortly. Very helpful for visualizing the processor activity and for debugging modified versions of the design. Requires installation of Tcl/Tk on your system. Invoked with the `-g` command line option. Running in GUI mode is only possible from within the directory (seq) in which the executable simulator program is located.

Several options from the command line:

- h** Prints a summary of all of the command line options.
- g** Run the simulator in GUI mode (the default is TTY mode).
- t** (TTY mode only) Runs both the processor and the ISA simulators, comparing the resulting values of the memory, register file, and condition codes. If no discrepancies are found, it prints the message 'ISA Check Succeeds.' Otherwise, it prints information about the words of the register file or memory that differ.
- l m** (TTY mode only) Sets the instruction limit, executing at most m instructions before halting (the default limit is 10000 instructions).
- v n** (TTY mode only) Sets the verbosity level to n, which must be between 0 and 2 with a default value of 2.

If running in GUI mode, It need to install Tcl/Tk alongwith the Tcl and Tk developer's packages.

(5) In **seq** directory, use following command to open GUI:

```
# make clean
```

```
# make ssim VERSION=std
```

```
# ./ssim -g ../y86-code/asum.yo
```

This code runs SSIM in GUI mode, executing the instructions in object code file *asum.yo* from the *y86-code* subdirectory.



Figure 2 Main control panel for SEQ simulator

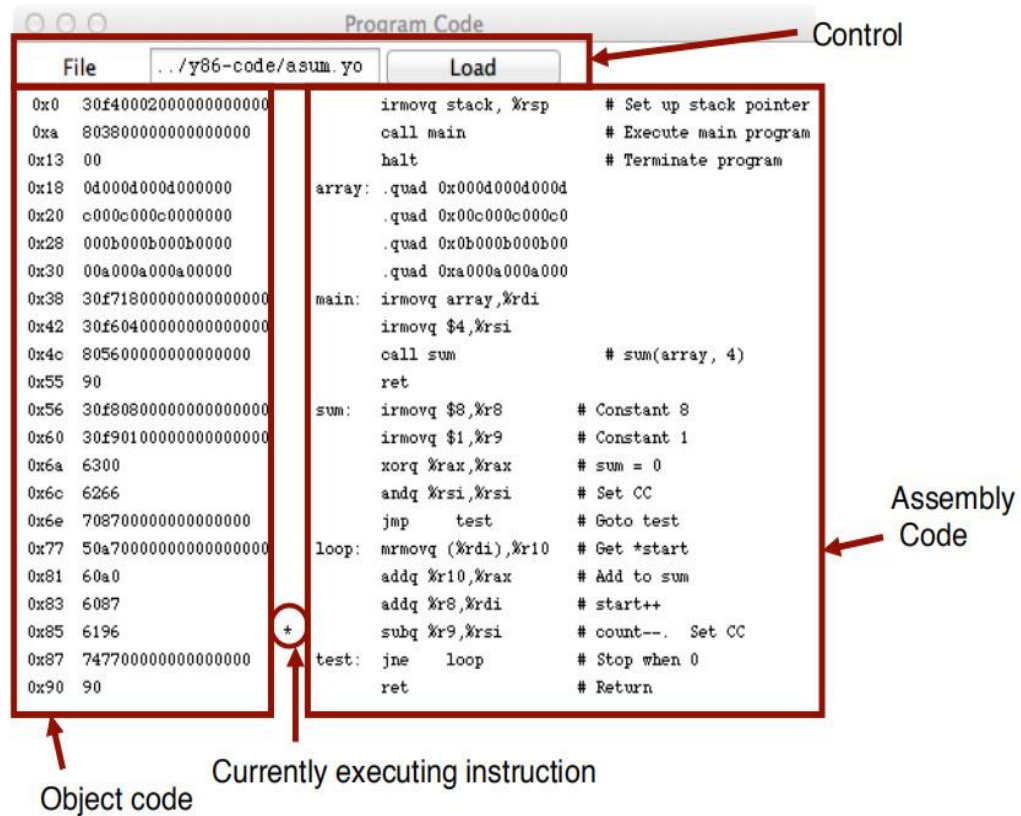


Figure 3 Code display window for SEQ simulator

	0x---0	0x---4	0x---8	0x---c
0x00f-	14	4	100	11
0x00e-	0	0	f8	3d

0x00e0 0x00e4 0x00e8 0x00ec

Figure 4 Memory display window for SEQ simulator

The main control window (Figure 2) contains buttons to control the simulator as well as status information about the state of the processor. The different parts of the window are labeled in the figure:

Control: The buttons along the top control the simulator. Clicking the

Quit button causes the simulator to exit. Clicking the *Go* button causes the simulator to start running. Clicking the *Stop* button causes the simulator to stop temporarily. Clicking the *Step* button causes the simulator to execute one instruction and then stop. Clicking the *Reset* button causes the simulator to return to its initial state.

The slider below the buttons controls the speed of the simulator when it is running. Moving it to the right makes the simulator run faster.

Stage values: This part of the display shows the values of the different processor signals during the current instruction evaluation. The main difference is that the simulator displays the name of the instruction in a field labeled *Instr*, rather than the numeric values of *icode* and *ifun*. Similarly, all register identifiers are shown using their names, rather than their numeric values, with “----” indicating that no register access is required.

Register file: This section displays the values of the 15 program registers. The register that has been updated most recently is shown highlighted in light blue. Register contents are not displayed until after the first time they are set to non-zero values.

Remember that when an instruction writes to a program register, the register file is not updated until the beginning of the next clock cycle. This means that you must step the simulator one more time to

see the update take place.

Stat: This shows the status of the current instruction being executed. The possible values are:

AOK: No problem encountered.

ADR: An addressing error has occurred either trying to read an instruction or trying to read or write data. Addresses cannot exceed *0x0FFF*.

INS: An illegal instruction was encountered.

HLT: A *halt* instruction was encountered.

Condition codes: These show the values of the three condition codes: *ZF*, *SF*, and *OF*.

Remember that when an instruction changes the condition codes, the condition code register is not updated until the beginning of the next clock cycle. This means that you must step the simulator one more time to see the update take place.

Example:

The processor state illustrated in Figure 2 is for the first execution of line 29 of the *asum.yo* program shown in Figure 1. We can see that the program counter is at *0x085*, that it has processed the instruction *addq %r8, %rdi*, that register *%rax* holds *0xd000d000d*, the sum of the first array element, and *%rsi* holds 4, the count that is about to be decremented. Register *%rdi* holds *0x020*, the address of these cond array

element. There is a pending write of *0x03* to register *%rsi* (since *dstE* is set to *%rsi* and *valE* is set to *0x03*). This write will take place at the start of the next clock cycle.

The window depicted in Figure 3 shows the object code file that is being executed by the simulator. The editbox identifies the file name of the program being executed. You can edit the file name in this window and click the Load button to load a new program. The left hand side of the display shows the object code being executed, while the right hand side shows the text from the assembly code file. The center has an asterisk(*) to indicate which instruction is currently being simulated. This corresponds to line 29 of the *asum.yo* program shown in Figure 1.

The window shown in Figure 4 shows the contents of the memory. It shows only those locations between the minimum and maximum addresses that have changed since the program began executing. Each row shows the contents of two memory words. Thus, each row shows 16 bytes of the memory, where the addresses of the bytes differ in only their least significant hexadecimal digits. To the left of the memory values is the “root” address, where the least significant digit is shown as “-”. Each column then corresponds to words with least significant address digits *0x0*, and *0x8*. The example shown in Figure 4 has arrows indicating memory locations *0x01f0* and *0x01f8*.

The memory contents illustrated in the figure show the stack

contents of the *asum.yo* program shown in Figure 1 during the execution of the *sum* procedure. Looking at the stack operations that have taken place so far, we see that *%rsp* was initialized to *0x200* (line 3). The call to *main* on line 4 pushes the return pointer *0x013*, which is written to address *0x01f8*. Procedure *main* calls *sum* (line 16), causing the return pointer *0x055* to be written to address *0x01f0*. That accounts for all of the words shown in this memory display, and for the stack pointer being set to *0x01f0*.

(6) The work should be done in directory **sim/misc** in this part. The task is to simulate the following two Y86-64 programs. The required behavior of these programs is defined by the example C functions in *sim/misc/examples.c*.

(6.1) **sum.yo**: Iteratively sum linked list elements

Simulate a Y86-64 program **sum.yo** that iteratively sums the elements of a linked list. It consists of some code that sets up the stack structure, invokes a function, and then halts. In this case, the function should be Y86-64 code for a function (*sum list*) that is functionally equivalent to the C *sum list* function in Figure 1.

(6.2) **rsum.yo**: Recursively sum linked list elements

Simulate a Y86-64 program *rsum.yo* that recursively sums the elements of a linked list. It is similar to the code in *sum.yo*, except that it uses a function *rsum list* that recursively sums a list of numbers.

(7) Testing the program. The Y86-64 assembler takes a Y86-64 assembly code file with extension **.ys** and generates a file with extension **.yo**. Put **rsum.ys** and **sum.ys** into **misc** directory. Use command:

```
# make sum.yo
```

```
# make rsum.yo
```

And then run the program in **seq** directory using:

```
# ./ssim -g ../misc/sum.yo
```

```
# ./ssim -g ../misc/rsum.yo
```

(8) For the programs **sum.ys** and **rsum.ys**, the result will be shown in register **%rax** with the sum of **0xcba**.

(9) Explain the function and processor state of each step in the source file of **sum.ys** and **rsum.ys** according to your snapshot.

3. Lab Device or Environment

Ubuntu 16.04 (64-bit) on virtual machine (VMware workstation 17 Pro).

4. Results and Analysis

a) Preparation

The modified *sources.list* (from website in section 3) and its backups are as following:

```
sources.list
/etc/apt

deb http://cn.archive.ubuntu.com/ubuntu/ xenial main restricted universe
multiverse
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-security main restricted
universe multiverse
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-updates main restricted universe
multiverse
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-backports main restricted
universe multiverse

##測試版源
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-proposed main restricted
universe multiverse{#{if:1}

# 源碼
deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial main restricted universe
multiverse
deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-security main restricted
universe multiverse
deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-updates main restricted
universe multiverse
deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-backports main restricted
universe multiverse

##測試版源
deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-proposed main restricted
universe multiverse}#{if:1}

# Canonical 合作夥伴和附加
deb http://archive.canonical.com/ubuntu/ xenial partner
deb http://extras.ubuntu.com/ubuntu/ xenial main}
```

```
sources.list.bak
/etc/apt

#deb cdrom:[Ubuntu 16.04.7 LTS _Xenial Xerus_ - Release amd64 (20200806)]/
xenial main restricted

# See http://help.ubuntu.com/community/UpgradeNotes for how to upgrade to
# newer versions of the distribution.
deb http://cn.archive.ubuntu.com/ubuntu/ xenial main restricted
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial main restricted

## Major bug fix updates produced after the final release of the
## distribution.
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-updates main restricted
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-updates main restricted

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team. Also, please note that software in universe WILL NOT receive any
## review or updates from the Ubuntu security team.
deb http://cn.archive.ubuntu.com/ubuntu/ xenial universe
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial universe
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-updates universe
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-updates universe

## N.B. software from this repository is ENTIRELY UNSUPPORTED by the Ubuntu
## team, and may not be under a free licence. Please satisfy yourself as to
## your rights to use the software. Also, please note that software in
## multiverse WILL NOT receive any review or updates from the Ubuntu
## security team.
deb http://cn.archive.ubuntu.com/ubuntu/ xenial multiverse
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial multiverse
deb http://cn.archive.ubuntu.com/ubuntu/ xenial-updates multiverse
# deb-src http://cn.archive.ubuntu.com/ubuntu/ xenial-updates multiverse
```

Installation of corresponding tools:

```

honor@honor-virtual-machine:~$ flex -V
flex 2.6.0
honor@honor-virtual-machine:~$ bison -V
bison (GNU Bison) 3.0.4
由 Robert Corbett 和 Richard Stallman 编写。

版权所有 (C) 2015 Free Software Foundation, Inc.
本程序是自由软件；请参看源代码的版权声明。本软件不提供任何担保；
包括没有适销性和某一专用目的下的适用性担保。
honor@honor-virtual-machine:~$ tclsh <<< 'puts [info patchlevel]'
8.6.5
honor@honor-virtual-machine:~$ wish <<< 'puts [info patchlevel]'
8.6.5
honor@honor-virtual-machine:~$ dpkg -l | grep -E 'tcl.*dev|tk.*dev'
ii  tcl8.5-dev:amd64      8.5.19-1
                                amd64      Tcl (the Tool Command Language) v8.5 - development f
files
ii  tk8.5-dev:amd64      8.5.19-1ubuntu1
                                amd64      Tk toolkit for Tcl and X11, v8.5 - development files
honor@honor-virtual-machine:~$ cd ./文档/Lab_5/sim
honor@honor-virtual-machine:~/文档/Lab_5/sim$ ls -l misc/yas misc/yis seq/ssim
-rwxrwxr-x 1 honor honor 102744 12月  5 14:54 misc/yas
-rwxrwxr-x 1 honor honor  62136 12月  5 14:54 misc/yis
-rwxrwxr-x 1 honor honor  46744 12月  5 14:58 seq/ssim

```

Modification of **Makefile** in **sim** and **seq**:

```

# Modify the following line so that gcc can find the libtcl.so and
# libtk.so libraries on your system. You may need to use the -L option
# to tell gcc which directory to look in. Comment this out if you
# don't have Tcl/Tk.

TKLIBS=-L/usr/lib -ltk8.5 -ltcl8.5

# Modify the following line so that gcc can find the tcl.h and tk.h
# header files on your system. Comment this out if you don't have
# Tcl/Tk.

TKINC=-isystem /usr/include/tcl8.5

# Modify the following line so that gcc can find the libtcl.so and
# libtk.so libraries on your system. You may need to use the -L option
# to tell gcc which directory to look in. Comment this out if you
# don't have Tcl/Tk.

TKLIBS=-L/usr/lib -ltk8.5 -ltcl8.5

# Modify the following line so that gcc can find the tcl.h and tk.h
# header files on your system. Comment this out if you don't have
# Tcl/Tk.

TKINC=-isystem /usr/include/tcl8.5

```

b) Building of Y86-64 Tools


```

honor@honor-virtual-machine:~/文档/Lab_5/sim$ make clean
rm -f *~ core
(cd misc; make clean)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/misc'
rm -f *.o *.yo *.exe yis yas hcl2c mux4 *~ core.*
rm -f hcl.tab.c hcl.tab.h lex.yy.c yas-grammar.c
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/misc'
(cd pipe; make clean)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/pipe'
rm -f psim pipe-*.c *.o *.exe *~
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/pipe'
(cd seq; make clean)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/seq'
rm -f ssim ssim+ seq*~.c *.o *~ *.exe *.yo *.ys
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/seq'
(cd y86-code; make clean)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/y86-code'
rm -f *.o *.yis *~ *.yo *.pipe *.seq *.seq+ core
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/y86-code'
(cd ptest; make clean)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/ptest'
rm -f *.o *~ *.yo *.ys
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/ptest'

```

```

honor@honor-virtual-machine:~/文档/Lab_5/sim$ make
(cd misc; make all)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/misc'
gcc -Wall -O1 -g -c yis.c
gcc -Wall -O1 -g -c isa.c
gcc -Wall -O1 -g yis.o isa.o -o yis
gcc -Wall -O1 -g -c yas.c
flex yas-grammar.lex
mv lex.yy.c yas-grammar.c
gcc -O1 -c yas-grammar.c
gcc -Wall -O1 -g yas-grammar.o yas.o isa.o -lfl -o yas
bison -d hcl.y
flex hcl.lex
gcc -O1 node.c lex.yy.c hcl.tab.c outgen.c -o hcl2c
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/misc'
(cd pipe; make all GUIMODE= TKLIBS="-L/usr/lib -ltk8.5 -ltcl8.5" TKINC="-isystem
/usr/include/tcl8.5")
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/pipe'
# Building the pipe-std.hcl version of PIPE
../misc/hcl2c -n pipe-std.hcl < pipe-std.hcl > pipe-std.c
gcc -Wall -O2 -isystem /usr/include/tcl8.5 -I../misc -o psim psim.c pipe-std.c
\
    ../misc/isa.c -L/usr/lib -ltk8.5 -ltcl8.5 -lm
./gen-driver.pl -n 4 -f ncopy.ys > sdriver.ys
../misc/yas sdriver.ys
./gen-driver.pl -n 63 -f ncopy.ys > ldriver.ys
../misc/yas ldriver.ys
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/pipe'
(cd seq; make all GUIMODE= TKLIBS="-L/usr/lib -ltk8.5 -ltcl8.5" TKINC="-isystem
/usr/include/tcl8.5")
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/seq'
# Building the seq-std.hcl version of SEQ
../misc/hcl2c -n seq-std.hcl <seq-std.hcl >seq-std.c

```



```

gcc -Wall -O2 -isystem /usr/include/tcl8.5 -I../misc -o ssim \
    seq-std.c ssim.c ../misc/isa.c -L/usr/lib -ltk8.5 -ltcl8.5 -lm
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/seq'
(cd y86-code; make all)
make[1]: Entering directory '/home/honor/文档/Lab_5/sim/y86-code'
../misc/yas abs-asum-cmov.yas
../misc/yas abs-asum-jmp.yas
../misc/yas asum.yas
../misc/yas asumr.yas
../misc/yas asumi.yas
../misc/yas cjr.yas
../misc/yas j-cc.yas
../misc/yas poptest.yas
../misc/yas pushquestion.yas
../misc/yas pushtest.yas
../misc/yas prog1.yas
../misc/yas prog2.yas
../misc/yas prog3.yas
../misc/yas prog4.yas
../misc/yas prog5.yas
../misc/yas prog6.yas
../misc/yas prog7.yas
../misc/yas prog8.yas
../misc/yas prog9.yas
../misc/yas prog10.yas
../misc/yas ret-hazard.yas
make[1]: Leaving directory '/home/honor/文档/Lab_5/sim/y86-code'

```

c) Explanation of *sum.yo*

The screenshot displays the Y86-64 Processor simulator interface. The main window is titled "Y86-64 Processor: seq-std.hcl". It features a control panel with buttons for "Quit", "Go", "Stop", "Step", and "Reset". Below the control panel, the "Simulator Speed (10*log Hz)" is set to 0. The "Processor State" section shows the current state of the processor, including the "newPC" (000000000000000A), "valM" (0000000000000000), "valE" (0000000000000100), and "valA" (0000000000000000). The "Execute Stage" is highlighted, showing the instruction "irmovq %rsp, %rsp" being executed. The "Decode Stage" and "Fetch Stage" are also visible. The "Register File" section shows the values of the registers: %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, and %r14. The "Program Code" section shows the assembly code for the sum.yo program, starting with "irmovq Stack, %rsp". The "Memory Contents" section shows the memory at address 0x---0, containing the value 0x---8.

Initially, the command *irmovq Stack,%rsp* stores the immediate number *Stack* into the stack pointer *%rsp*. Then the stack pointer points to 0x100.

The screenshot shows the Y86-64 Processor simulator interface. The top panel includes controls for Quit, Go, Stop, Step, and Reset, along with a Simulator Speed slider set to 0. The main panel displays the Processor State, which is divided into several stages: PC Update Stage, Memory Stage, Execute Stage, Decode Stage, Fetch Stage, and Register File. The PC is currently 0000000000000014. The Register File shows %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, and %r14. The %rsp register is highlighted in blue and contains the value 100. The Condition Codes are Z=1, S=0, O=0.

The Program Code panel on the right shows the assembly code for the file `../misc/sum.yo`. The code starts with `irmovq Stack, %rsp` and `irmovq ele1, %rdi`, which are the instructions currently being executed.

Then, *irmovq ele1 %rdi* stores the immediate number, the address of the first node, *ele1* into the register *%rdi*.

The screenshot shows the Y86-64 Processor simulator interface after the execution of the `call sum_list` instruction. The PC is now 0000000000000050. The Register File shows that the %rdi register is now highlighted in blue and contains the value 20. The Condition Codes are Z=1, S=0, O=0.

The Program Code panel on the right shows the assembly code for the file `../misc/sum.yo`. The code starts with `irmovq Stack, %rsp` and `irmovq ele1, %rdi`, which are the instructions currently being executed.

call sum_list calls the function *sum*, and store the data of linked list nodes including address and corresponding values by

commands *.quad*. Each command increases the PC by 1 byte. And *%rsp* decreases by 1 byte to store the address 0x1d to return and points to the entry of the function. Here the compiler optimizes the process of leaf function *sum_list* so that there is no more operations related to stack.

The screenshot displays the Y86-64 Processor simulator interface. The top control panel includes buttons for 'Quit', 'Go', 'Stop', 'Step', and 'Reset', along with a 'Simulator Speed' slider set to 0. The main window is divided into several sections: 'Processor State' showing 'newPC' at 0000000000000052 and 'valM' at 0000000000000000; 'PC Update Stage'; 'Memory Stage' showing 'Cnd' as 'N' and 'valE' as 0000000000000000; 'Execute Stage' showing 'valA' at 0000000000000000, 'valB' at 0000000000000000, 'dstE' as '%rax', 'dstM' as '---', 'srcA' as '%rax', and 'srcB' as '%rax'; 'Decode Stage' showing 'Instr' as 'xorq', 'rA' as '%rax', 'rB' as '%rax', 'valC' at 0000000000000000, and 'valP' at 0000000000000052; 'Fetch Stage'; and 'Register File' showing registers %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, %r14, and %c00. The 'Condition Codes' are Z=1, S=0, O=0. The 'Program Code' window shows assembly instructions for a function named 'sum_list', including stack pointer manipulation, loop setup, and return. The 'Memory Contents' window shows a memory location 0x00f- with values 0 and 1d.

The command *xorq %rax %rax* initialize the local variable *val* stored in *%rax* to 0.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC: 0000000000000054

PC Update Stage

valM: 0000000000000000

Memory Stage

Cnd: valE: N 0000000000000020

Execute Stage

valA: 0000000000000020 valB: 0000000000000020 dstE: %rdi dstM: %rdi srcA: %rdi srcB: %rdi

Decode Stage

Instr: andq rA: %rdi rB: %rdi valC: 0000000000000000 valP: 0000000000000054

Fetch Stage

PC: 0000000000000052

Register File

%rax	%rcx	%rdx	%rbx	%rsp
0				
%rbp	%rsi	%rdi	%r8	%r9
		20		
%r10	%r11	%r12	%r13	%r14
c00				

Stat: AOK Condition Codes: Z 0 S 0 O 0

Program Code

File: ../misc/sum.yo Load

```

0x0 30f400010000000000000000 irmovq Stack,%rax
0xa 30f720000000000000000000 irmovq ele1, %rdi
0x14 805000000000000000000000 call sum_list
0x1d 00 halt
0x20 0a0000000000000000000000 .quad 0x00a
0x28 300000000000000000000000 .quad ele2
0x30 b00000000000000000000000 .quad 0x0b0
0x38 400000000000000000000000 .quad ele3
0x40 000c00000000000000000000 .quad 0xc00
0x48 000000000000000000000000 .quad 0
0x50 6300 xorq %rax,%rax # val = 0
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 737e00000000000000000000 * je done # Yes, goto done
0x5d 50a700000000000000000000 loop: mrmovq (%rdi),%r10 # t = ls->val
0x67 507708000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 60a0 addq %r10,%rax # val += t
0x73 6277 andq %rdi,%rdi # Check ls
0x75 745d00000000000000000000 jne loop # If null, goto done
0x7e 90 done: ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d

The command `andq %rdi, %rdi` judges whether `ele1`, or `ls`, the address of the first node, is zero and correspondingly adjust the condition codes ZF. Here it is not NULL thus ZF is zero.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC: 000000000000005D

PC Update Stage

valM: 0000000000000000

Memory Stage

Cnd: valE: N 0000000000000000

Execute Stage

valA: 0000000000000000 valB: 0000000000000000 dstE: %rdi dstM: %rdi srcA: %rdi srcB: %rdi

Decode Stage

Instr: je rA: %rdi rB: %rdi valC: 000000000000007E valP: 000000000000005D

Fetch Stage

PC: 0000000000000054

Register File

%rax	%rcx	%rdx	%rbx	%rsp
0				
%rbp	%rsi	%rdi	%r8	%r9
		20		
%r10	%r11	%r12	%r13	%r14
c00				

Stat: AOK Condition Codes: Z 0 S 0 O 0

Program Code

File: ../misc/sum.yo Load

```

0x0 30f400010000000000000000 irmovq Stack,%rax
0xa 30f720000000000000000000 irmovq ele1, %rdi
0x14 805000000000000000000000 call sum_list
0x1d 00 halt
0x20 0a0000000000000000000000 .quad 0x00a
0x28 300000000000000000000000 .quad ele2
0x30 b00000000000000000000000 .quad 0x0b0
0x38 400000000000000000000000 .quad ele3
0x40 000c00000000000000000000 .quad 0xc00
0x48 000000000000000000000000 .quad 0
0x50 6300 xorq %rax,%rax # val = 0
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 737e00000000000000000000 * je done # Yes, goto done
0x5d 50a700000000000000000000 loop: mrmovq (%rdi),%r10 # t = ls->val
0x67 507708000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 60a0 addq %r10,%rax # val += t
0x73 6277 andq %rdi,%rdi # Check ls
0x75 745d00000000000000000000 jne loop # If null, goto done
0x7e 90 done: ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d

`je done` operates jumps to the operation done if the condition code ZF

is 1. Here it is zero thus it does not jump.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC: 0000000000000067

PC Update Stage

valM: 000000000000000A

Memory Stage

Cnd: N valE: 0000000000000020

Execute Stage

valA: 0000000000000000 valB: 0000000000000020 dstE: %r10 dstM: %rdi srcA: srcB:

Decode Stage

Instr: mrmovq rA: %r10 rB: %rdi valC: 0000000000000000 valP: 0000000000000067

Fetch Stage

PC: 0000000000000067

Register File

%rax	0	%rcx		%rdx		%rbx		%rsp	f8
%rbp		%rsi		%rdi	20	%r8		%r9	
%r10		%r11		%r12		%r13		%r14	
c00									

Stat: AOK Condition Codes: Z=0 S=0 O=0

Program Code

File: ../misc/sum.yo Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq ele1,%rdi
0x14 80500000000000000000000000000000 call sum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad ele2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad ele3
0x40 00c00000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 6300 xorq %rax,%rax # val = 0
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 737e0000000000000000000000000000 je done # Yes, goto done
0x5d 50a70000000000000000000000000000 * loop: mrmovq (%rdi),%r10 # t = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 60a0 addq %r10,%rax # val += t
0x73 6277 andq %rdi,%rdi # Check ls
0x75 745d0000000000000000000000000000 jne loop # If null, goto done
0x7e 90 done: ret # return
  
```

Memory Contents

0x---0 0x---8

0x00f- 0 1d

The first command in loop moves the value (*%rdi*), the value of *ele1* stored in memory into register *%r10*.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC: 0000000000000071

PC Update Stage

valM: 0000000000000030

Memory Stage

Cnd: N valE: 0000000000000028

Execute Stage

valA: 0000000000000000 valB: 0000000000000020 dstE: %rdi dstM: %rdi srcA: srcB:

Decode Stage

Instr: mrmovq rA: %rdi rB: %rdi valC: 0000000000000008 valP: 0000000000000071

Fetch Stage

PC: 0000000000000067

Register File

%rax	0	%rcx		%rdx		%rbx		%rsp	f8
%rbp		%rsi		%rdi	20	%r8		%r9	
%r10		%r11		%r12		%r13		%r14	
a									

Stat: AOK Condition Codes: Z=0 S=0 O=0

Program Code

File: ../misc/sum.yo Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq ele1,%rdi
0x14 80500000000000000000000000000000 call sum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad ele2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad ele3
0x40 00c00000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 6300 xorq %rax,%rax # val = 0
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 737e0000000000000000000000000000 je done # Yes, goto done
0x5d 50a70000000000000000000000000000 * loop: mrmovq (%rdi),%r10 # t = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 60a0 addq %r10,%rax # val += t
0x73 6277 andq %rdi,%rdi # Check ls
0x75 745d0000000000000000000000000000 jne loop # If null, goto done
0x7e 90 done: ret # return
  
```

Memory Contents

0x---0 0x---8

0x00f- 0 1d

The next command moves the value *Mem[reg[%rdi]+8]* into the

register *%rbi*, that is, move the address of next node, *ele2* into *%rbi*,
ls.

The screenshot displays the Y86-64 Processor simulator interface. The top panel shows the Processor State with various stages: PC Update Stage, Memory Stage, Execute Stage, Decode Stage, and Fetch Stage. The Register File section shows the state of registers, with %r10 highlighted. The Program Code panel on the right shows the assembly code for sum.yo, including instructions like `irmovq Stack,%rsp`, `irmovq ele1,%rdi`, `call sum_list`, and `halt`. The Memory Contents panel at the bottom right shows the memory at address 0x00f0.

Then the add command add the value in *%r10* and the value in *%rax*,
that is the value of *ele1* and *val*, and store the value after addition
into *%rax*, that is to renew the value of *val*.

The screenshot shows the Y86-64 Processor simulator interface. The top panel includes controls for Quit, Go, Stop, Step, and Reset, along with a Simulator Speed slider set to 0. The main panel displays the Processor State, which is divided into several stages: PC Update Stage, Memory Stage, Execute Stage, Decode Stage, Fetch Stage, and Register File. The PC is currently 0000000000000075. The Register File shows %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, and %r14. The Condition Codes are Z=0, S=0, O=0.

The Program Code panel shows the assembly code for the file `../misc/sum.yo`. The code includes instructions for setting up a stack frame, calculating the sum of elements in an array, and returning the result. The first instruction is `irmovq Stack, %rsp`.

The Memory Contents panel shows the memory address 0x00f- with a value of 0, and 0x---8 with a value of 1d.

The next command check whether *ls* is NULL again and to set the condition code ZF. Here *ele2* is not NULL thus ZF is 0.

The screenshot shows the Y86-64 Processor simulator interface after the first instruction has been executed. The Processor State panel shows the PC updated to 000000000000005D. The Register File shows %rdi updated to 30. The Condition Codes are Z=0, S=0, O=0. The Program Code panel shows the next instruction, `jne loop`, which checks the ZF flag and jumps to the `loop` label if it is not equal to 1 (i.e., if ZF is 0). The Memory Contents panel shows the memory address 0x00f- with a value of 0, and 0x---8 with a value of 1d.

jne loop jumps to loop if the condition code ZF is 0. Here ZF is zero thus the loop continues. After one loop *val* equals to *a*.

The loop of handling *ele2* and *ele3* is similar thus omitting the process of *ele2* and *ele3* is omitted.

When the third loop will end, that is, the next of *ele3* is NULL, the *andq* sets ZF as 1. Then the *jne* loop are done and return the value.

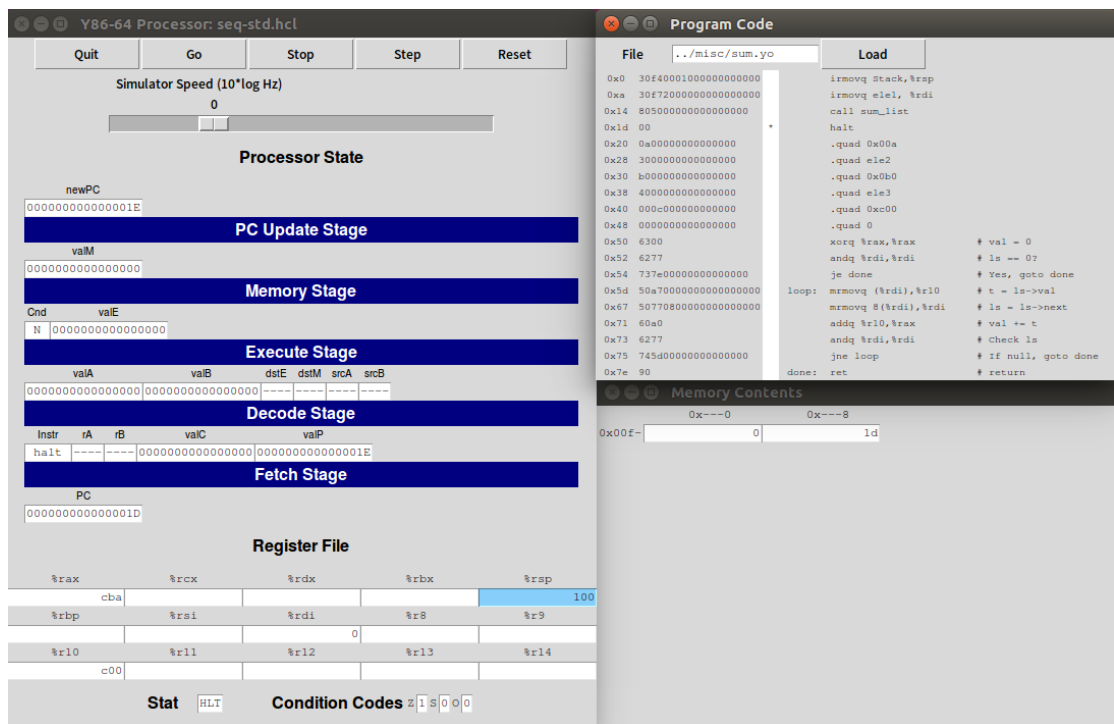
The stack pointer points to 0x100 again. Then execute *halt* ends the programme.

The result *val* is stored in *%rax* as 0xcba, consistent with expected result.

The screenshot displays the Y86-64 Processor simulator interface. The top control panel includes buttons for Quit, Go, Stop, Step, and Reset, along with a Simulator Speed (10*log Hz) slider set to 0. The main window is divided into several sections:

- Processor State:** Shows the newPC (000000000000007E), vaIM (0000000000000000), and Cnd (N) with vaIE (0000000000000000).
- PC Update Stage:** A blue bar indicating the current stage.
- Memory Stage:** A blue bar indicating the current stage.
- Execute Stage:** A blue bar indicating the current stage.
- Decode Stage:** A blue bar indicating the current stage.
- Fetch Stage:** A blue bar indicating the current stage.
- Register File:** A table showing the state of registers. The %rax register contains the value cba (0xcba), and the %rsp register contains f8 (0xf8).
- Program Code:** A list of assembly instructions with their addresses. The instructions include *irmovq*, *call*, *halt*, *addq*, *andq*, *jne*, and *ret*.
- Memory Contents:** A table showing the state of memory. The address 0x00f- contains the value 0, and the address 0x---8 contains the value 1d.

The status bar at the bottom shows the Stat (ACK) and Condition Codes (ZF=1, SF=0, OF=0).



d) Explanation of *rsum.yo*

The screenshot shows the Y86-64 Processor simulator interface. The top panel includes controls for Quit, Go, Stop, Step, and Reset, along with a Simulator Speed slider set to 0 (10*log Hz). The main display area is divided into several sections:

- Processor State:** Shows the newPC register at 000000000000000A.
- PC Update Stage:** The valM register is 0000000000000000.
- Memory Stage:** The Cnd register is N and the valE register is 0000000000000100.
- Execute Stage:** The valA register is 0000000000000000, valB is 0000000000000000, dstE is %rsp, dstM is ----, srcA is ----, and srcB is ----.
- Decode Stage:** The Instr register is irmovq, rA is ----, rB is %rsp, valC is 0000000000000100, and valP is 000000000000000A.
- Fetch Stage:** The PC register is 0000000000000000.
- Register File:** A table showing registers %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, and %r14.
- Stat:** AOK
- Condition Codes:** Z=1, S=0, O=0, C=0.

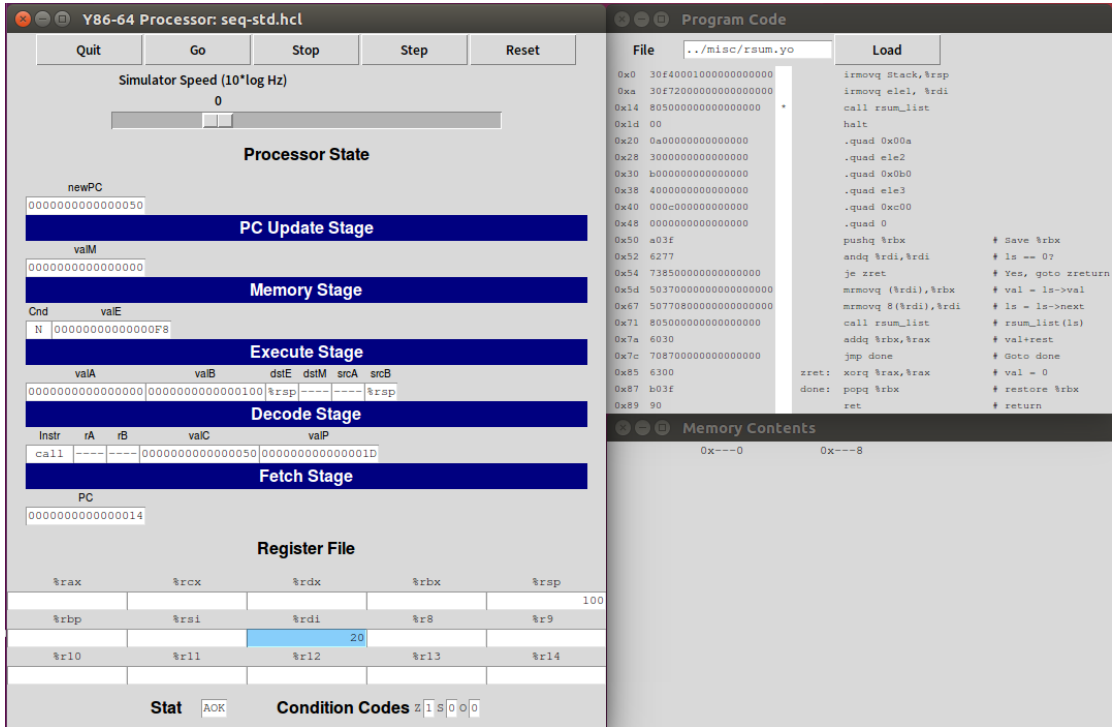
The right panel shows the Program Code for the file `../misc/rsum.yo`. The first instruction is `irmovq Stack,%rsp`, which is highlighted. The code continues with `irmovq ele1,%rdi`, `call rsum_list`, `halt`, and several `.quad` directives.

Similar to *sum.yo*, initially the command `irmovq Stack,%rsp` stores the immediate number *Stack* into the stack pointer `%rsp`. Then the stack pointer points to 0x100. Then, `irmovq ele1,%rdi` stores the immediate number, the address of the first node, *ele1* into the register `%rdi`.

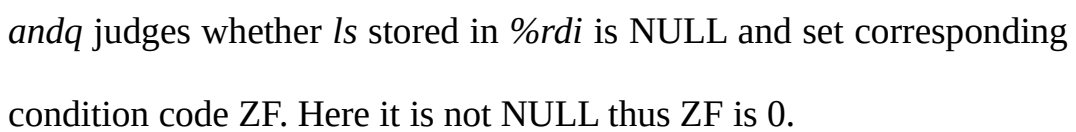
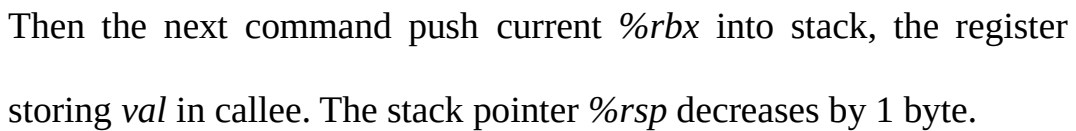
The screenshot shows the Y86-64 Processor simulator interface after the first two instructions have been executed. The main display area is divided into several sections:

- Processor State:** Shows the newPC register at 0000000000000014.
- PC Update Stage:** The valM register is 0000000000000000.
- Memory Stage:** The Cnd register is N and the valE register is 0000000000000020.
- Execute Stage:** The valA register is 0000000000000000, valB is 0000000000000000, dstE is %rdi, dstM is ----, srcA is ----, and srcB is ----.
- Decode Stage:** The Instr register is irmovq, rA is %rdi, rB is 0000000000000020, valC is 0000000000000014, and valP is 0000000000000014.
- Fetch Stage:** The PC register is 000000000000000A.
- Register File:** A table showing registers %rax, %rcx, %rdx, %rbx, %rsp, %rbp, %rsi, %rdi, %r8, %r9, %r10, %r11, %r12, %r13, and %r14. The %rsp register is highlighted in blue and contains the value 100.
- Stat:** AOK
- Condition Codes:** Z=1, S=0, O=0, C=0.

The right panel shows the Program Code for the file `../misc/rsum.yo`. The first two instructions, `irmovq Stack,%rsp` and `irmovq ele1,%rdi`, are highlighted. The code continues with `call rsum_list`, `halt`, and several `.quad` directives.



When calling the function *rsum_list*, the data of linked list nodes including address and corresponding values are stored by commands *.quad*. Each command increases the PC by 1 byte. And *%rsp* decreases by 1 byte to store the address 0x1d to return and points to the entry of the function.



Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
000000000000005D

PC Update Stage

valM
0000000000000000

Memory Stage

Cnd valE
N 0000000000000000

Execute Stage

valA valB dstE dstM srcA srcB
0000000000000000 0000000000000000 ----

Decode Stage

Instr rA rB valC valP
je ---- 0000000000000085 000000000000005D

Fetch Stage

PC
0000000000000054

Register File

%rax	%rcx	%rdx	%rbx	%rsp
				f0
%rbp	%rsi	%rdi	%r8	%r9
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 0 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad e1e2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad e1e3
0x40 000c0000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 73850000000000000000000000000000 je zret # Yes, goto zreturn
0x5d 50370000000000000000000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

Memory Contents

0x---0 0x---8
0x00f- 0 1d

The *je* jumps to *zret* when ZF is 1. Here it is zero thus the programme continue sequentially.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
0000000000000067

PC Update Stage

valM
0000000000000000A

Memory Stage

Cnd valE
N 000000000000000020

Execute Stage

valA valB dstE dstM srcA srcB
0000000000000000 0000000000000020 ---- %rbx ---- %rdi

Decode Stage

Instr rA rB valC valP
mrmovq %rbx,%rdi 0000000000000000 0000000000000067

Fetch Stage

PC
000000000000005D

Register File

%rax	%rcx	%rdx	%rbx	%rsp
				f0
%rbp	%rsi	%rdi	%r8	%r9
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 0 S 0 O 0

Program Code

File ../misc/rsum.yo Load

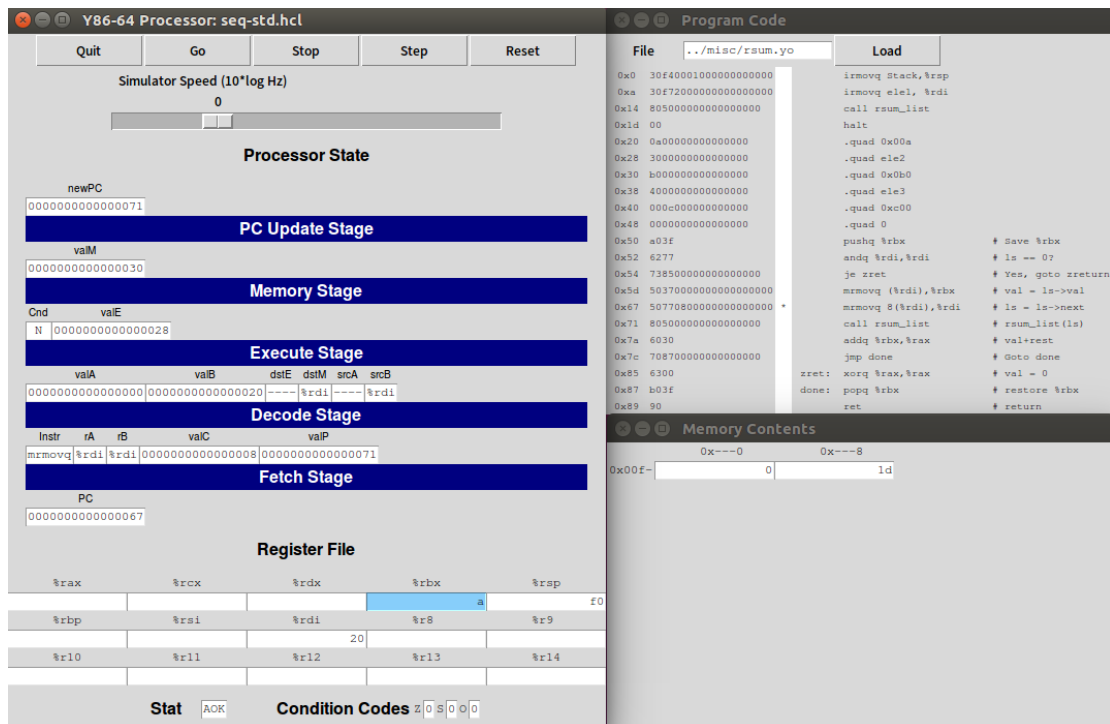
```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad e1e2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad e1e3
0x40 000c0000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 73850000000000000000000000000000 je zret # Yes, goto zreturn
0x5d 50370000000000000000000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

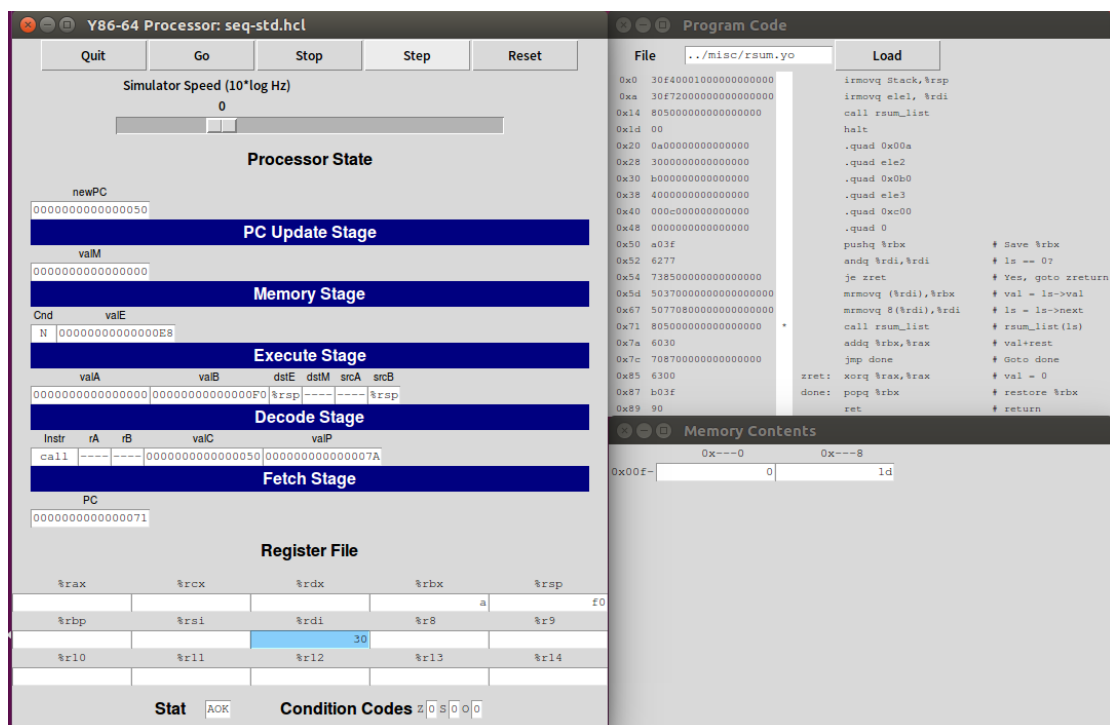
Memory Contents

0x---0 0x---8
0x00f- 0 1d

Then the command move the value in memory pointed by *%rdi*, that is *ls->val*, into the register *%rbx*, that is *val*.



The next command moves the next of *ls* into *%rbi*. That is to renew *ls* to its next.



Then the programme calles the *rsum_list* function recursively. Decrease the *%rsp* by 1 byte to store the return address 0x7a of this

recursion. Then the PC is updated to the starting address of *rsum_list* function.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
00000000000000052

PC Update Stage

valM
00000000000000000

Memory Stage

Cnd valE
N 000000000000000E0

Execute Stage

valA valB dstE dstM srcA srcB
000000000000000A 000000000000000E8 %rsp ---- %rbx %rsp

Decode Stage

Instr rA rB valC valP
pushq %rbx ---- 0000000000000000 00000000000000052

Fetch Stage

PC
00000000000000050

Register File

%rax	%rcx	%rdx	%rbx	%rsp
			a	e8
%rbp	%rsi	%rdi	%r8	%r9
		30		
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 0 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq e1e1, %rdi
0x14 80500000000000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad e1e2
0x30 b0000000000000000000000000000000 .quad 0xb0
0x38 40000000000000000000000000000000 .quad e1e3
0x40 00c00000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 a03f * pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # Is == 0?
0x54 73850000000000000000000000000000 je zret # Yes, goto xreturn
0x5d 50370000000000000000000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000000000000000 jmp done # Goto done
0x85 6300 rret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 rret: ret # return

```

Memory Contents

0x--0	0x--8
0x00f- 0	1d
0x00e- 0	7a

Then the next steps are similar to former steps, push current *%rbx*, or *val* into stack to store the state of caller, check whether the node is NULL, take the value of this node, renew *ls* to its next, call the function again, renew the stack pointer and then store return address until the node is NULL. And the stack pointer is updated to 0xc0.

At the last invoking, the *andq* set ZF as 1 since the node is NULL.

The screenshot shows the Y86-64 Processor simulator interface. The top panel includes controls for Quit, Go, Stop, Step, and Reset, along with a Simulator Speed slider set to 0 (10*log Hz). The main display area is divided into several sections:

- Processor State:** Shows the newPC register at 0000000000000085.
- PC Update Stage:** Shows the valM register at 0000000000000000.
- Memory Stage:** Shows the Cnd register at Y and the valE register at 0000000000000000.
- Execute Stage:** Shows the valA register at 0000000000000000, the valB register at 0000000000000000, the dstE register at 0000000000000000, the dstM register at 0000000000000000, the srcA register at 0000000000000000, and the srcB register at 0000000000000000.
- Decode Stage:** Shows the Instr register at je, the rA register at 0000000000000085, the rB register at 000000000000005D, the valC register at 0000000000000000, and the valP register at 0000000000000000.
- Fetch Stage:** Shows the PC register at 0000000000000054.
- Register File:** Shows the %rax register at 0000000000000000, the %rcx register at 0000000000000000, the %rdx register at 0000000000000000, the %rbx register at 0000000000000000, the %rsp register at 0000000000000000, the %rbp register at 0000000000000000, the %rsi register at 0000000000000000, the %rdi register at 0000000000000000, the %r8 register at 0000000000000000, the %r9 register at 0000000000000000, the %r10 register at 0000000000000000, the %r11 register at 0000000000000000, the %r12 register at 0000000000000000, the %r13 register at 0000000000000000, and the %r14 register at 0000000000000000.
- Stat:** Shows the AOK status.
- Condition Codes:** Shows the ZF flag at 1, the SF flag at 0, the OF flag at 0, and the CF flag at 0.

The right panel shows the Program Code window with the File dropdown set to ../misc/rsum.yo and the Load button. The code window displays the following assembly code:

```

0x0 30f400010000000000000000    irmovq Stack,%rsp
0xa 30f720000000000000000000    irmovq ele1,%rdi
0x14 805000000000000000000000    call rsum_list
0x1d 00                             halt
0x20 0a0000000000000000000000    .quad 0x00a
0x28 300000000000000000000000    .quad ele2
0x30 b00000000000000000000000    .quad 0x0b0
0x38 400000000000000000000000    .quad ele3
0x40 000c00000000000000000000    .quad 0xc00
0x48 000000000000000000000000    .quad 0
0x50 a03f                          pushq %rbx          # Save %rbx
0x52 6277                          andq %rdi,%rdi      # Is == 0?
0x54 738500000000000000000000    je zret            # Yes, goto zreturn
0x5d 503700000000000000000000    mrmovq (%rdi),%rbx # val = ls->val
0x67 507708000000000000000000    mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 805000000000000000000000    call rsum_list      # rsum_list(ls)
0x7a 6030                          addq %rbx,%rax       # val+rest
0x7c 708700000000000000000000    jmp done            # Goto done
0x85 6300                          zret: xorq %rax,%rax # val = 0
0x87 b03f                          done: popq %rbx      # restore %rbx
0x89 90                             ret                  # return

```

The Memory Contents window shows the following memory addresses and values:

0x---	0x---
0x00f-	0
0x00e-	a
0x00d-	b0
0x00c-	c00

Then, the command *je zret* jumps to *zret* since ZF is 1.

The screenshot shows the Y86-64 Processor simulator interface after the execution of the *je zret* instruction. The main display area is divided into several sections:

- Processor State:** Shows the newPC register at 0000000000000087.
- PC Update Stage:** Shows the valM register at 0000000000000000.
- Memory Stage:** Shows the Cnd register at N and the valE register at 0000000000000000.
- Execute Stage:** Shows the valA register at 0000000000000000, the valB register at 0000000000000000, the dstE register at %rax, the dstM register at %rax, the srcA register at %rax, and the srcB register at %rax.
- Decode Stage:** Shows the Instr register at xorq, the rA register at %rax, the rB register at %rax, the valC register at 0000000000000000, and the valP register at 0000000000000087.
- Fetch Stage:** Shows the PC register at 0000000000000085.
- Register File:** Shows the %rax register at 0000000000000000, the %rcx register at 0000000000000000, the %rdx register at 0000000000000000, the %rbx register at 0000000000000000, the %rsp register at 0000000000000000, the %rbp register at 0000000000000000, the %rsi register at 0000000000000000, the %rdi register at 0000000000000000, the %r8 register at 0000000000000000, the %r9 register at 0000000000000000, the %r10 register at 0000000000000000, the %r11 register at 0000000000000000, the %r12 register at 0000000000000000, the %r13 register at 0000000000000000, and the %r14 register at 0000000000000000.
- Stat:** Shows the AOK status.
- Condition Codes:** Shows the ZF flag at 1, the SF flag at 0, the OF flag at 0, and the CF flag at 0.

The right panel shows the Program Code window with the File dropdown set to ../misc/rsum.yo and the Load button. The code window displays the same assembly code as the previous screenshot.

The Memory Contents window shows the same memory addresses and values as the previous screenshot.

The next command return the *%rax*, *val* of callee, as 0.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
0000000000000089

PC Update Stage

valM
00000000000000C0

Memory Stage

Cnd valE
N 00000000000000C8

Execute Stage

valA valB dstE dstM srcA srcB
00000000000000C0 00000000000000C0 %rbx %rbx %rsp %rsp

Decode Stage

Instr rA rB valC valP
popq %rbx ---- 0000000000000000 0000000000000089

Fetch Stage

PC
0000000000000087

Register File

%rax	%rcx	%rdx	%rbx	%rsp
0			c00	c0
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 1 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f40001000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000 call rsum_list
0x1d 00 halt
0x20 0a0000000000000000 .quad 0x00a
0x28 300000000000000000 .quad e1e2
0x30 b00000000000000000 .quad 0x0b0
0x38 400000000000000000 .quad e1e3
0x40 000c00000000000000 .quad 0xc00
0x48 000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 738500000000000000 je zret # Yes, goto xreturn
0x5d 503700000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 507708000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 805000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 708700000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return

```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

Then the programme pops the value, that is the *val* of caller storing in *%rbx*, in the stack and then store it into *%rbx*. Then the stack pointer is updated to 0xc8 by increasing 1 byte.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
000000000000007A

PC Update Stage

valM
000000000000007A

Memory Stage

Cnd valE
N 00000000000000D0

Execute Stage

valA valB dstE dstM srcA srcB
00000000000000C8 00000000000000C8 %rsp ---- %rsp %rsp

Decode Stage

Instr rA rB valC valP
ret ---- 0000000000000000 000000000000008A

Fetch Stage

PC
0000000000000089

Register File

%rax	%rcx	%rdx	%rbx	%rsp
0			c00	c8
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 1 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f40001000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000 call rsum_list
0x1d 00 halt
0x20 0a0000000000000000 .quad 0x00a
0x28 300000000000000000 .quad e1e2
0x30 b00000000000000000 .quad 0x0b0
0x38 400000000000000000 .quad e1e3
0x40 000c00000000000000 .quad 0xc00
0x48 000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 738500000000000000 je zret # Yes, goto xreturn
0x5d 503700000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 507708000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 805000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 708700000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return

```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

Then *ret* ends the process of this function and the stack pointer

increases to 0xd0. The PC returns to 0x7a, the address to be returned stored in the stack.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
000000000000007C

PC Update Stage
valM
0000000000000000

Memory Stage
Cnd valE
N 00000000000000C00

Execute Stage
valA valB dstE dstM srcA srcB
00000000000000C00 0000000000000000 %rax ---- %rbx %rax

Decode Stage
Instr rA rB valC valP
addq %rbx,%rax 0000000000000000 000000000000007C

Fetch Stage
PC
000000000000007A

Register File

%rax	%rcx	%rdx	%rbx	%rsp
0			c00	d0
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat ACK Condition Codes Z 1 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f40001000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000 call rsum_list
0x1d 00 halt
0x20 0a0000000000000000 .quad 0x00a
0x28 300000000000000000 .quad e1e2
0x30 b00000000000000000 .quad 0x0b0
0x38 400000000000000000 .quad e1e3
0x40 00c000000000000000 .quad 0xc00
0x48 000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # Is == 0?
0x54 73850000000000000000 je zret # Yes, goto zreturn
0x5d 50370000000000000000 rrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000 rrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 zret # return

```

Memory Contents

0x00f-	0	1d
0x00e-	a	7a
0x00d-	b0	7a
0x00c-	c00	7a

Then the *addq* adds the sum of return value of callees, that is *rest*, storing in *%rax* and the value of current node storing in *%rbx*. Then store the result into *%rax*.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
0000000000000087

PC Update Stage
valM
0000000000000000

Memory Stage
Cnd valE
Y 0000000000000000

Execute Stage
valA valB dstE dstM srcA srcB
0000000000000000 0000000000000000 --- --- ---

Decode Stage
Instr rA rB valC valP
jmp --- 0000000000000087 0000000000000085

Fetch Stage
PC
000000000000007C

Register File

%rax	%rcx	%rdx	%rbx	%rsp
c00			c00	d0
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat ☒ AOK Condition Codes Z 0 S 0 O 0 D 0

Program Code

File Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad e1e2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad e1e3
0x40 000c0000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 73850000000000000000000000000000 je zret # Yes, goto zreturn
0x5d 50370000000000000000000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

Then jump to *done* again.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
0000000000000089

PC Update Stage
valM
0000000000000080

Memory Stage
Cnd valE
N 0000000000000008

Execute Stage
valA valB dstE dstM srcA srcB
0000000000000000 0000000000000000 %rsp %rbx %rsp %rsp

Decode Stage
Instr rA rB valC valP
popq %rbx --- 0000000000000000 0000000000000089

Fetch Stage
PC
0000000000000087

Register File

%rax	%rcx	%rdx	%rbx	%rsp
c00			c00	d0
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat ☒ AOK Condition Codes Z 0 S 0 O 0 D 0

Program Code

File Load

```

0x0 30f40001000000000000000000000000 irmovq Stack,%rsp
0xa 30f72000000000000000000000000000 irmovq e1e1,%rdi
0x14 80500000000000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a000000000000000000000000000000 .quad 0x00a
0x28 30000000000000000000000000000000 .quad e1e2
0x30 b0000000000000000000000000000000 .quad 0x0b0
0x38 40000000000000000000000000000000 .quad e1e3
0x40 000c0000000000000000000000000000 .quad 0xc00
0x48 00000000000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # ls == 0?
0x54 73850000000000000000000000000000 je zret # Yes, goto zreturn
0x5d 50370000000000000000000000000000 mrmovq (%rdi),%rbx # val = ls->val
0x67 50770800000000000000000000000000 mrmovq 8(%rdi),%rdi # ls = ls->next
0x71 80500000000000000000000000000000 call rsum_list # rsum_list(ls)
0x7a 6030 addq %rbx,%rax # val+rest
0x7c 70870000000000000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

Then the command pops the stack, the *val* of caller storing in *%rbx*, and stores it into *%rbx* again. The stack pointer is updated to 0xd8.

The screenshot shows the Y86-64 Processor simulator interface. The top panel includes control buttons (Quit, Go, Stop, Step, Reset) and a Simulator Speed slider set to 0. The main area displays the Processor State, which is divided into several stages: PC Update Stage, Memory Stage, Execute Stage, Decode Stage, Fetch Stage, and Register File. The PC is currently 000000000000007A. The Register File shows %rax as c00, %rbx as b0, and %rsp as d8. The Program Code panel on the right shows the first instruction: `irmovq Stack, %rsp`. The Memory Contents panel shows the stack pointer at 0x00f- with value 0.

Then return and updates the stack pointer.

The screenshot shows the Y86-64 Processor simulator interface after the first instruction has been executed. The PC is now 000000000000007C. The Register File shows %rax as c00, %rbx as b0, and %rsp as e0. The Program Code panel on the right shows the second instruction: `irmovq %eax, %rdi`. The Memory Contents panel shows the stack pointer at 0x00f- with value 0.

Add the value of the sum of callees stored in `%rax` and the `val` of current node stored in `%rbx`. Store the result into `%rax`.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
0000000000000087

PC Update Stage
valM
0000000000000000

Memory Stage
Cnd valE
Y 0000000000000000

Execute Stage
valA valB dstE dstM srcA srcB
0000000000000000 0000000000000000 ----

Decode Stage
Instr rA rB valC valP
jmp ---- 0000000000000087 0000000000000085

Fetch Stage
PC
000000000000007C

Register File

%rax	%rcx	%rdx	%rbx	%rsp
cb0			b0	e0
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 0 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f400010000000000000000 irmovq Stack,%rsp
0xa 30f720000000000000000000 irmovq %eax,%rdi
0x14 805000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a0000000000000000000000 .quad 0x00a
0x28 300000000000000000000000 .quad %eax
0x30 b00000000000000000000000 .quad 0x0b0
0x38 400000000000000000000000 .quad %eax
0x40 000c00000000000000000000 .quad 0xc00
0x48 000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # %rs == 0?
0x54 738500000000000000000000 je zret # Yes, goto zreturn
0x5d 503700000000000000000000 rmmovq (%rdi),%rbx # %val = %rs->%val
0x67 507708000000000000000000 rmmovq 8(%rdi),%rdi # %rs = %rs->%next
0x71 805000000000000000000000 call rsum_list # rsum_list(%rs)
0x7a 6030 addq %rbx,%rax # %val+rest
0x7c 708700000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # %val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

Jump to done again and repeat similar steps to add the values recursively until the stack pointer points to 0xf8.

Then, the last *ret* is operated and then the PC is updated to 0x1d and the stack pointer is updated to 0x100. The program halts.

Y86-64 Processor: seq-std.hcl

Quit Go Stop Step Reset

Simulator Speed (10*log Hz)
0

Processor State

newPC
000000000000001D

PC Update Stage
valM
000000000000001D

Memory Stage
Cnd valE
N 00000000000000100

Execute Stage
valA valB dstE dstM srcA srcB
000000000000000f8 000000000000000f8 %rsp ---- %rsp %rsp

Decode Stage
Instr rA rB valC valP
ret ---- 0000000000000000 000000000000008A

Fetch Stage
PC
0000000000000099

Register File

%rax	%rcx	%rdx	%rbx	%rsp
cba			0	f8
%rbp	%rsi	%rdi	%r8	%r9
		0		
%r10	%r11	%r12	%r13	%r14

Stat AOK Condition Codes Z 0 S 0 O 0

Program Code

File ../misc/rsum.yo Load

```

0x0 30f400010000000000000000 irmovq Stack,%rsp
0xa 30f720000000000000000000 irmovq %eax,%rdi
0x14 805000000000000000000000 call rsum_list
0x1d 00 halt
0x20 0a0000000000000000000000 .quad 0x00a
0x28 300000000000000000000000 .quad %eax
0x30 b00000000000000000000000 .quad 0x0b0
0x38 400000000000000000000000 .quad %eax
0x40 000c00000000000000000000 .quad 0xc00
0x48 000000000000000000000000 .quad 0
0x50 a03f pushq %rbx # Save %rbx
0x52 6277 andq %rdi,%rdi # %rs == 0?
0x54 738500000000000000000000 je zret # Yes, goto zreturn
0x5d 503700000000000000000000 rmmovq (%rdi),%rbx # %val = %rs->%val
0x67 507708000000000000000000 rmmovq 8(%rdi),%rdi # %rs = %rs->%next
0x71 805000000000000000000000 call rsum_list # rsum_list(%rs)
0x7a 6030 addq %rbx,%rax # %val+rest
0x7c 708700000000000000000000 jmp done # Goto done
0x85 6300 zret: xorq %rax,%rax # %val = 0
0x87 b03f done: popq %rbx # restore %rbx
0x89 90 ret # return
  
```

Memory Contents

0x---0	0x---8
0x00f- 0	1d
0x00e- a	7a
0x00d- b0	7a
0x00c- c00	7a

The screenshot displays the Y86-64 Processor simulator interface. The main window is titled "Y86-64 Processor: seq-std.hcl". It features a control bar with buttons: Quit, Go, Stop, Step, and Reset. Below this is a "Simulator Speed (10*log Hz)" slider set to 0. The "Processor State" section shows the current state of the processor, including the PC (000000000000001E), valM (0000000000000000), and the Register File. The Register File shows %rax containing 0xcba, which is the result of the sum. The "Program Code" panel on the right shows the assembly code being executed, including instructions like "irmovq Stack,%rsp", "irmovq %eax,%rdi", "call rsum_list", and "halt". The "Memory Contents" panel at the bottom right shows the state of memory at various addresses, including 0x00f, 0x00e, 0x00d, and 0x00c.

The result value is stored in %rax as 0xcba, consistent with expected result.

5. Appendix (Program Code)

sum.c:

```
/* sum_list - Sum the elements of a linked list */
long sum_list(list_ptr Ls)
{
    long val = 0;
    while (Ls)
    {
        val += Ls->val;
        Ls = Ls->next;
    }
    return val;
}
```

rsum.c:

```
/* rsum_list - Recursive version of sum_list */
long rsum_list(list_ptr Ls)
{
    if (!Ls)
```

```

    return 0;
else
{
    long val = ls->val;
    long rest = rsum_list(ls->next);
    return val + rest;
}
}

```

sum.yo:

	# Initial code
0x000: 30f4000100000000000000	irmovq Stack,%rsp
0x00a: 30f7200000000000000000	irmovq ele1,%rdi
0x014: 8050000000000000000000	call sum_list
0x01d: 00	halt
	# Sample linked list
0x020:	.align 8
0x020:	ele1:
0x020: 0a00000000000000000000	.quad 0x00a
0x028: 3000000000000000000000	.quad ele2
0x030:	ele2:
0x030: b000000000000000000000	.quad 0x0b0
0x038: 4000000000000000000000	.quad ele3
0x040:	ele3:
0x040: 000c000000000000000000	.quad 0xc00
0x048: 0000000000000000000000	.quad 0
	# long sum_list(list_ptr ls)
	# ls in %rdi
0x050:	sum_list:
0x050: 6300	xorq %rax,%rax # val = 0
0x052: 6277	andq %rdi,%rdi # ls == 0?
0x054: 737e000000000000000000	je done # Yes, goto done
0x05d: 50a7000000000000000000	loop: mrmovq (%rdi),%r10 # t = ls->val
0x067: 5077080000000000000000	mrmovq 8(%rdi),%rdi # ls = ls->next
0x071: 60a0	addq %r10,%rax # val += t
0x073: 6277	andq %rdi,%rdi # Check ls
0x075: 745d000000000000000000	jne loop # If null, goto done
0x07e: 90	done: ret # return
0x100:	.pos 0x100
0x100:	Stack:

rsum.yo:


```

0x000: 30f400010000000000000000 | # Initial code
0x00a: 30f720000000000000000000 |   irmovq Stack,%rsp
0x014: 805000000000000000000000 |   irmovq ele1, %rdi
0x01d: 00 |   call rsum_list
|   halt
|
| # Sample linked list
|   .align 8
0x020: |   ele1:
0x020: 0a0000000000000000 |       .quad 0x00a
0x028: 300000000000000000 |       .quad ele2
0x030: |   ele2:
0x030: b00000000000000000 |       .quad 0x0b0
0x038: 400000000000000000 |       .quad ele3
0x040: |   ele3:
0x040: 000c00000000000000 |       .quad 0xc00
0x048: 000000000000000000 |       .quad 0
|
| # long rsum_list(list_ptr ls)
| # ls in %rdi
0x050: |   rsum_list:
0x050: a03f |       pushq %rbx | # Save %rbx
0x052: 6277 |       andq %rdi,%rdi | # ls == 0?
0x054: 73850000000000000000 |       je zret | # Yes, goto zreturn
0x05d: 5037000000000000000000 |       mrmovq (%rdi),%rbx | # val = ls->val
0x067: 5077080000000000000000 |       mrmovq 8(%rdi),%rdi | # ls = ls->next
0x071: 8050000000000000000000 |       call rsum_list | # rsum_list(ls)
0x07a: 6030 |       addq %rbx,%rax | # val+restS|
0x07c: 7087000000000000000000 |       jmp done | # Goto done
0x085: 6300 |   zret: xorq %rax,%rax | # val = 0
0x087: b03f |   done: popq %rbx | # restore %rbx
0x089: 90 |       ret | # return
|
0x100: |   .pos 0x100
0x100: |   Stack:

```